Introduction & Problem Solving by Searching

Maciej Piasecki

Contact:
Building D21, room 230
e-mail: maciej.piasecki@pwr.edu.pl
clarin-pl.eu
pllum.org.pl
ai.pwr.edu.pl
tel.: +48 71 320 42 21

Office hours:
Fri. 8:00-10:00

Course outline: aims

1. To acquaint students with widely understood Artificial Intelligence: origins, goals, development directions.
2. To present currently most popular methods and techniques,
3. but also broader bases to improve understanding and offer choice.
4. To make aware of dangers and draw attention to responsibility:
   - ethics,
   - Explainable AI (XAI) and control.

- Course materials on ePortal: https://eportal.pwr.edu.pl/course/view.php?id=5836

Course outline: content

1. Problem solving by searching
   - a reminder - unintelligent simulation of intelligence
   - the ultimate genesis of AI
   - heuristic searching methods
2. Constraint Satisfaction Problems (CSP)
   - definition, methods of solving them — an intelligent approach to puzzles
3. Adversarial search and game playing
   - game tree, Min-Max algorithm, Alpha-beta pruning
   - logical games as a test for AI and the first won battle (Deep Blue)
4. Planning algorithms and their applications
   - domain knowledge in searching — symbolic representation of operators
   - contemporary examples, e.g. applications in scheduling and resource allocation, combining Large Language Models (LLMs) with planning tools,

Course outline: content

5. Knowledge representation, knowledge bases, logical inference
   - information, data, knowledge
   - propositional logic and First Order Logic
   - semantic networks and ontologies, knowledge bases in AI systems
   - simple inference: Forward Chaining, Backward Chaining,
   - logical inference, unification, Prolog and resolution
   - non-logical inference: distance and similarity
6. Introduction to Machine Learning
   - the concept, paradigms
   - supervised ML - approaches, selected examples, e.g. Decision Trees,
   - statistical learning - Naive Bayes algorithm and statistical knowledge
   - datasets and evaluation of ML-based systems
7. Unsupervised ML
   - from data to knowledge
   - similarity: Case-based Reasoning, Instance-based Learning
   - clustering
8. Neural Networks
   - artificial neuron
   - Multilayer Perceptron (MLP)
   - a brief overview of Deep Neural Networks (Deep Learning)

Course outline: content (if time left)

8. Neural Networks
   - artificial neuron
   - Multilayer Perceptron (MLP)
   - a brief overview of Deep Neural Networks (Deep Learning)
9. NLP (Natural Language Processing) and introduction to LLMs (Large Language Models)
   - universal means of communication and information exchange
   - language representation and typical tasks, e.g. text classification, Information Extraction, QA, MT, ASR
   - Large Language Models and Generative AI
   - LLM use, e.g. in context learning and prompt engineering
   - LLM development: from data to models aligned to human preferences
10. Reinforcement Learning and example applications, e.g. AlphaGo OpenAI Gym
11. LLM-based systems
    - RAG, generative agents, reasoning in LLMs, neurosymbolic systems
    - selected aspects of the evaluation of LLMs
12. Legal and ethical aspects of AI, eXplainable AI (XAI)
13. Fuzzy logic, uncertain and imprecise knowledge processing
    - probabilistic logic, Bayesian networks
    - fuzzy inference and other approaches

Course outline: evaluation

- Laboratory part — a summary, the full version on ePortal
  - in short: five assignments with deadlines, each worth 50 or 100 pts.
  - each missed deadline costs 20% of points for a given assignment, deducted per a lab class of the delay
  - all assignments must be presented
  - minimum 50% to pass
- Lecture
  - exam during the examination period
  - in a physical form, on-site
  - extra bonus points resulting from discussions may be collected during lectures
    - asking questions
    - answering to lecturer's questions
    - making remarks that are relevant to the lecture content

Literature – general

- Wolfgang Ertel. Introduction to Artificial Intelligence. Springer, 2017. (available via WUST network)
- Jurafsky, D. & Martin, J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics and Speech Recognition Prentice Hall, 2000; 3 edition (draft 2021): https://web.stanford.edu/~jurafsky/slp3/
- Miroslav Kubat. An Introduction to Machine Learning. Springer, 2017. (available via WUST network)
- Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, Introduction to Information Retrieval, Cambridge University Press. 2008. https://nlp.stanford.edu/IR-book/information-retrieval-book.html
- David L. Poole and Alan K. Mackworth. Artificial Intelligence: Foundations of Computational Agents, 2nd Edition, Cambridge University Press, 2017, http://artint.info/2e/html/ArtInt2e.html
- Russel S.J., Norvig Peter, Artificial Intelligence. A Modern Approach. Prentice Hall Series in Artificial Intelligence, 1995 (or a newer issue)
  - Chapter 5 (a new version): http://aima.cs.berkeley.edu/newchap05.pdf
- Sandro Skansi. Introduction to Deep Learning. From Logical Calculus to Artificial Intelligence. Springer, 2018. (available via WUST network)

Literature – supplementary

- Rodrigo Fernandes de Mello and Moacir Antonelli Ponti. Machine Learning — A Practical Approach on the Statistical Learning Theory. Springer, 2018. (available via WUST network)
- Robert Johansson. Numerical Python. Apress, 2019. (available via WUST network and Springer)
- Umberto Michelucci. Advanced Applied Deep Learning. Apress, 2019. (available via WUST network and Springer)
- Donald J. Norris. Beginning Artificial Intelligence with the Raspberry Pi. Apress, 2017. (available via WUST network and Springer)
- Mitchell Tom M., Machine Learning. McGraw-Hill companies, Inc., 1997.
- Richard S. Sutton and Andrew G. Barto. Reinforcement Learning: An Introduction. The MIT Press, 2020.
- Charu C. Aggarwal. Artificial Intelligence. A Textbook. Springer. 2021.

Dreams about Artificial Intelligence

- When have the humanity started dreaming about artificial intelligence?

Dreams about Artificial Intelligence

- When did humanity start dreaming about Artificial Intelligence?
- At the very beginning of our culture!

Dreams about Artificial Intelligence

„Ever since Homer wrote of mechanical "tripods" waiting on the gods at dinner, imagined mechanical assistants have been a part of our culture. However, only in the last half century have we, the AI community, been able to build experimental machines that test hypotheses about the mechanisms of thought and intelligent behavior and thereby demonstrate mechanisms that formerly existed only as theoretical possibilities."

B. G. Buchanan (2005) A (very) brief history of artificial intelligence. AI Magazine **26** (4), pp. 53–60.
https://ojs.aaai.org//index.php/aimagazine/article/view/1848

- Greek myths: the history of Thalos - a bronze giant made by Hephaistos guarding Crete.
- Greek myths: tripods "in their own walking motion" cast by Hephaistos, mentioned by Homer.
- Han Dynasty in the 3rd century BC in the Middle Kingdom: an automatic orchestra played for the emperor.
- Sui Dynasty (581-618 AD): "A book of hydraulic elegance".

Adrienne Mayor. "Gods and Robots: Myths, Machines, and Ancient Dreams of Technology" following Anna Błońska z kopalniawiedzy.pl (access 25 II 20226)

https://kopalniawiedzy.pl/Adrienne-Mayor-mity-legendy-sztuczna-inteligencja-roboty-Hezjod-Homer-Talos-Pandora,29678

Threats about AI

"Success in creating AI might be the biggest event in human history.

Unfortunately it might also be the last, unless we learn how to avoid the risks."

Stephen Hawking — following (Aggarwal, 2021)

Ways of AI

- Two defining perspectives
  - construction of systems or models mimicking the human brain or way of thinking
  - development of systems or algorithms that express ability to solve problems difficult to humans with performance approaching the one of humans
- "The two schools of thought" (Aggarwal, 2021: pp. 2)
  - deduction vs induction
    - deduction: from the general to the specific, from rules (knowledge) and facts to conclusions, e.g.

      All canine animals have four legs. All dogs are canines. ⟹ *Therefore, dogs have four legs.*
    - induction: from the specific to the general, a kind of 'faulty' logic — generalising from a limited number of examples, e.g.
      Tomasz is a student. Dominik is a student. Both are well prepared for a laboratory class. ⟹ *Therefore, all students are well prepared for the class.*
  - reasoning vs learning
    - reasoning: knowledge base ⟹ chain of assertions ⟹ true conclusions
    - learning: statistical inference (generalisation, induction) ⟹ rules
      - e.g. *thousands of sea creatures lay eggs* ⟹ *all sea creatures are most likely to lay eggs* (what about whales! dolphins! seals! etc.)

Deductive reasoning methods

- "Reason about a base of known hypotheses, which are assumed to be a base of incontrovertible truths" (Aggarwal, 2021: pp. 3)
- Approaches
  - graph search — functions for state transitions
  - symbolic logic (algebraic methods) — inference or symbolic transformations
- Scheme
  - theory — a knowledge base
    - assertions (facts) and assumed hypotheses (typically rules)
  - logical inference (systematic, formal procedure)
    - reasoning about unknown facts or verifying new hypothesis
- Weaknesses
  - unable to inferences from incomplete evidence
  - all conclusions (new knowledge) are already somehow included in the knowledge base
    - they can be computed in advance
  - but facts change in applications
    - e.g. a medical system supporting doctors embodies verified knowledge in hundreds of rule, but patients' data change

Deductive reasoning methods: examples

- MYCIN
  - an *expert system* encoding medical knowledge in symbolic rules
  - including a *knowledge base* of bacteria and antibiotics
  - input: information collected by a physician
  - output: suggestions and *explanations* (important!)
- Deep Blue (IBM, 1998)
  - chess playing system (algorithms, knowledge base and computing hardware)
  - based on fast searching across possible game states
  - the first machine to defeat a human world champion
  - contemporary AlphaZero is based on Deep Reinforcement Learning
- Systran machine translation system
  - text transformation via large, manually crafted formal grammars
    - implemented as transducers (finite state machines)
  - knowledge base: grammars, lexicons, lexico-syntatic structures, translation memories
  - grammars supported by heuristics and searching

Inductive learning methods

- Learning algorithm, which makes a general hypothesis from a set of specific examples, e.g.
  - examples of many animals with descriptions, including different types of teeth
    - herbivores have different characteristics of teeth and claws, than carnivores
  - a function selecting carnivores as having sharp claws and long canine teeth
- A hypothesis — "statistical likelihoods relating the observed characteristics of the input data"
  - types, groups, similarities etc. of examples
  - e.g. Probability(herbivores) = f(Canine teeth length, Claw length)
- Machine learning algorithm to construct $f(.)$
  - start: a generic form (*prior assumption*) i.e. a basic model
  - learning specific details (e.g. parameters)
  - a hypothesis
    - represented by the final form of $f(.)$
    - „a mathematical function that *approximately maps* inputs to outputs"
    - e.g. based on probability that <Canine teeth length, Claw length> is associated with herbivores

Inductive learning methods: examples

- Vast majority of contemporary AI is based on Machine Learning
- Image recognition and understanding, speech recognition, machine translation, intelligent controllers (cars, planes) …
- Often AI = systems based on Machine Learning
- Challenges (selected from many):
  - trustworthiness
  - and explainability
  - learning from single examples or a few examples.

Combined: inductive and deductive schema

- Inductive learning:
  - creativity (in some sense), wide coverage, approximate behaviour for incomplete and imprecise data
- Deductive reasoning (knowledge-based symbolic systems)
  - transfer of verified knowledge into systems
  - partial control over the systems' behaviour
    - e.g. sanity check for critical decisions
  - improved explainability
- Neuro-symbolic approaches
  - different forms of combining symbolic knowledge, also logical rules, with Deep Neural Networks and Deep Learning
- Generative AI
  - large generative models for images and texts
  - unsupervised learning (autoregressive) + supervised (instructions) + Reinforcement Learning (alignment with preferences)
  - especially Large Language Models, trained to many tasks
  - act as a kind 'generative association memories'
  - prompted by case descriptions (in-context learning) - produce adapted solutions

The course map

- Deductive branch [~12 hours]
  - Problem solving by searching
  - Constraint Satisfaction Problems
  - Knowledge representation, knowledge bases, logical inference
  - Planning algorithms and their applications
  - Fuzzy logic, uncertain and imprecise knowledge processing
- Inductive branch [~13 hours]
  - Introduction to Machine Learning
  - Neural Networks
  - Unsupervised ML and Reinforcement Learning
- Combined approaches (but still dominated by inductive methods) [~5 hours]
  - Natural Language Processing (Natural Language Engineering)
  - Large Language Models (LLMs) and Generative AI
  - Reasoning in LLMs and neurosymbolic approaches (e.g. RAG = knowledge base + LLM)

Is this AI?

- AI as a way to simulate the human brain — strong AI
  - mind, problem-solving ability, thinking process, human being, …
  - cognitive psychology, biological inspiration
  - brain modelling
- AI as a way to solve problems difficult for humans
  - considered to require human intelligence or creativity
  - many NP-hard problems, too complex for direct computations
  - many may be perceived as optimisation problems,
    - e.g. task scheduling, travelling salesman
  - it is only the result and performance that matter, not the methods
    - i.e. we do not care for internal compatibility with the cognitive brain models
- The Turing Test
  - three terminals are physically separated
  - one questioner — human, and two respondents: human and machine
  - the questioner task: to identify a non-human interlocutor
  - problems (many), e.g. a smart machine may intentionally fail the test
  - contemporary chatbots are close to passing, if not already beyond
    - are they intelligent??

Solving problems

- Problem components
  - data collection of (the initial information), describing the problem
  - a set of operations (actions), which may be applied to come to a solution
  - goal – a description – what is a solution to the problem
    - a goal state: features or constraints
- To solve a problem, one needs:
  - a precise problem definition – the starting point (initial state)
  - knowledge representation (about the task)
  - selection of the most appropriate technology to solve the problem
- Intelligent agent solving problems difficult for humans
  - intelligent autonomous agent
  - perceptions $\Longrightarrow$ state modification
    - through sensors
  - state $\Longrightarrow$ actions
    - through actuators

Types of problems - environments

- Characteristics from the perspective of an *intelligent agent* solving a problem
- Accessible vs Inaccessible
  - accessible – in all aspects, complete information
  - inaccessible – in some or all aspects
- Deterministic vs Nondeterministic
  - deterministic – from the agent perspective
    - next state = the current state + action results
  - nondeterministic – complex, uncertainty
- Episodic vs Non-episodic
  - episodic – observations (experiences) can be divided into episodes
    - perception ⟹ acting
    - lack of dependency between episodes
    - evaluation of actions within episodes, no necessity to infer based on history
- Static vs Dynamic
  - static – no time flow
  - dynamic – the environment may change during action execution
- Discrete vs Continuous

Types of problem

- Single-state problem
  - fully accessible environment
  - action results are known and deterministic
    - for instance: Problem Solving by Searching
- Multiple-state problem
  - not fully accessible
  - action results are known (specified)
- Problem with the uncertainty aspects
  - in perception (state) or execution (actions)
  - interleaved searching and execution
  - experimenting and exploration
    - acquiring knowledge about actions
  - for instance: Reinforcement Learning, also Deep Reinforcement Learning

Problem Solving by Searching

- Single-state problem
- Accessible vs Inaccessible
  - accessible – in all aspects, complete information
  - inaccessible – in some or all aspects
- Deterministic vs Nondeterministic
  - deterministic — from the agent perspective
    - next state = the current state + action results
  - nondeterministic – complex, uncertainty
- Episodic vs Non-episodic
  - episodic – observations (experiences) can be divided into episodes
    - perception —> acting
    - lack of dependency between episodes
    - evaluation of actions within episodes, no necessity to infer on the basis of history
- Static vs Dynamic
  - static – no time flow
  - dynamic – the environment may change during action execution
- Discrete vs Continuous

Problem Solving by Searching

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action
  persistent: *seq*, an action sequence, initially empty
    state, some description of the current world state
    goal, a goal, initially null
    problem, a problem formulation
  state ← UPDATE-STATE(*state*,*percept*) **if** *seq* is empty **then**
    goal ← FORMULATE-GOAL(*state*)
    problem ←FORMULATE-PROBLEM(*state*,*goal*)
    seq ← SEARCH(*problem*)
    if *seq* = *failure* **then return** a null action
  action ← FIRST(*seq*)
  seq ← REST(*seq*) return *action*

(Russel & Norvig 1995, pp. 57)

Problem Solving by Searching

- Defined by Newell and Simon, a renowned and seminal book: Human Problem Solving (1972)
- Occurs in most AI tasks: planning, learning, etc.
- NP-complete or worse, there is often no 'clever' algorithm
- Problem solving can be formulated as a search task:
  - start – the current state of our world
  - end (goal) – the desired state, e.g. known by its properties
  - solution – a way of transforming the current state into the target one
- Challenge: to offer a solution general enough to be suitable for many applications
- Formal perspective: finding paths in a directed graph when given the starting and goal (test) nodes
- People use intuition to find solutions to difficult problems, but people do not solve overall problems; rather, the specific ones, about which they know more than just the search space
- In the computing methods, the additional knowledge is heuristic knowledge

Formal perspective

- Arc $\langle n_1, n_2 \rangle$ is an arc coming out from n_1 and entering to n_2
- n_2 is the neighbour of n_1 if there is an arc from n_1 to n_2;
  - (is arc $\langle n_1, n_2 \rangle \in A$),
  - it is not a symmetrical relation by default!
- The arc can be labelled
- The path from s to g:
  - a sequence of nodes $\langle n_0, n_1, \ldots, n_k \rangle$ such that $s = n_0$, $g = n_k$, and $\langle n_{i-1}, n_i \rangle \in A$;
  - (i.e., there is an arc from n_{i-1} to n_i for each i (a sequence of arcs,
    $\langle n_o, n_1 \rangle, \langle n_1, n_2 \rangle, \ldots, \langle n_{k-1}, n_k \rangle$, or a sequence of labels on these arcs)
- Cycle – a non-empty path and an end node is the same as the initial node: $\langle n_0, n_1, \ldots, n_k \rangle$ such that $n_0 = n_k$ and $k \neq 0$.

Formal perspective

- Directed acyclic graph (DAG)
- Tree:
  - DAG with one node without entering arc (root), each other node has exactly one arc entering, node without outgoing arcs — leaves
- Problem as a graph:
  - defining the start node and target nodes
  - a solution is a path from the start node to a target node

Formal perspective

- Cost of an arc – a positive number assigned to arc $cost(\langle n_i,n_j\rangle)$
- Cost of the path $p = \langle n_0, n_1,..., n_k\rangle$ - the total cost of arches:
  - $cost(p) = cost(\langle n_0,n_1\rangle) + ...+ cost(\langle n_{k-1},n_k\rangle)$
- The optimal solution – a path p with the lowest cost, i.e., there is no other path p' between the *start* and *goal* nodes, for which:
  - $cost(p')<cost(p)$
- Forward branching factor (FBF) of a given node – the number of arcs outgoing from the node
- Backward branching factor(BBF) – the number of arcs entering the node
- Tree: if for every node FBF = b, then there are b^n nodes with the distance equal to n arcs from any node

Components of the Space of States

- A set of states
- Marked out a subset of states called the initial states
- Actions (operators) on states
  - an action function: States → States
- Goal/target states
  - often given in the form of Boolean functions: goal(s)
- Acceptability criterion for solutions (quality measure)
  - for example, each action sequence that leads an agent to a target state may be acceptable, or there may be a cost associated with the actions, and the task may be to find a sequence of the minimal cost – the optimal solution, or it may be a satisfactory solution to the distance from the optimum 10%.
- For most of the AI problems:
  - the action sequence leading to a solution is not predefined and may be determined by a systematic analysis of alternatives
- The task of the search method is to select the best possible operator (according to the criteria)
- Search strategies: algorithmic realisation of search methods

General searching algorithm

```
function General-Search (problem, QUEUING-FN)
  //returns a solution, or a failure
{ nodes <-- MAKE-QUEUE(
            MAKE-NODE(INITIAL-STATE[problem]))
 loop {
   if nodes is empty then return failure
   node <-- REMOVE-FRONT(nodes)
   if GOAL-TEST[problem]
     applied to STATE(node) succeeds then return node
   nodes <-- QUEUING-FN(nodes,
            EXPAND(node, OPERATORS[problem]))
      }
}
```

General searching algorithm

- Independent of any search strategy
- Idea:
  - incrementally expands paths from the starting point
- Keeps the current front of explored nodes
- The front extends on non-explored nodes to the destination
- Problem: which nodes should be chosen for expansion?
- A node is tested whether it is a goal state or not when a given path is chosen to be expanded (not when it is added to the front)
- Expansion of a node from its parent that has no neighbours and is not a goal means removing it
- Search strategy determines which nodes to choose for expansion

Searching strategies

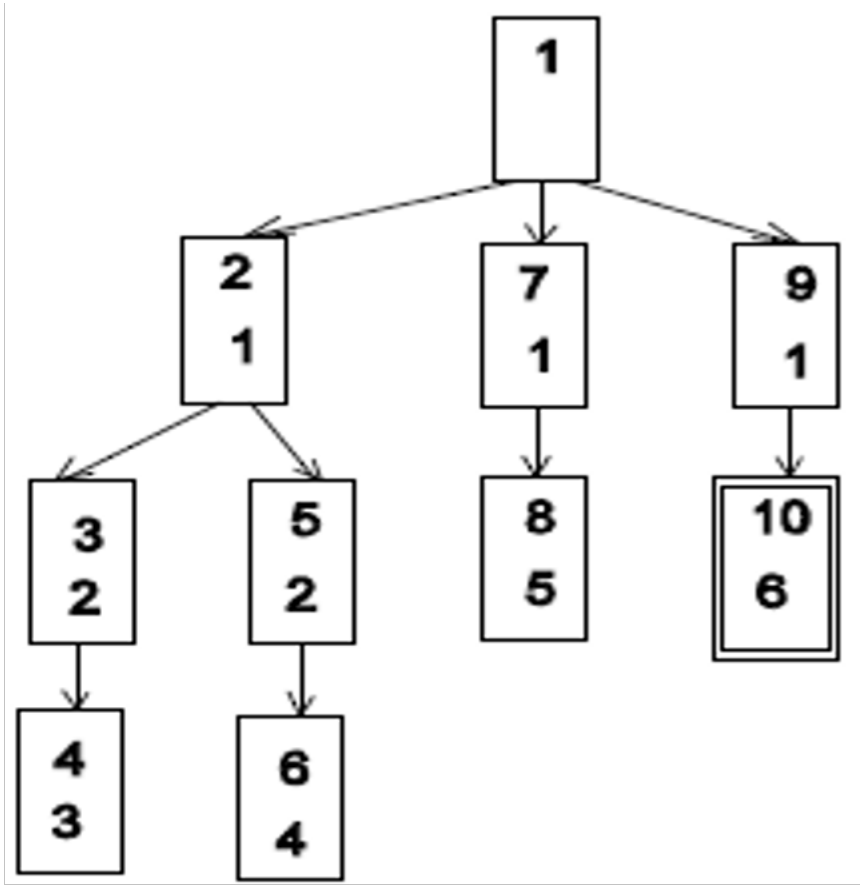
- Uninformed
  - Blind strategies
    - do not take into account the location of the target, not 'paying attention' to where, until it finds a goal node (they do not consider any information about the states and the goals to decide which path to expand first on the frontier; they do not take into account the specific nature of the problem)
  - Depth-First Search
    - front runs as a stack: last-in first-out: the last item added to the stack is first selected for expansion
  - Breadth-First Search
    - front is the FIFO queue (first-in, first-out)
  - Lowest-Cost-First Search
    - the simplest way to find the minimum cost path, similar to BFS, expands the path with the minimal cost, a priority queue defined by a cost function
- Informed (heuristic)
  - additional knowledge about the problem expressed by a heuristic (a heuristic function)
- Adversarial search: uncertainty caused by the second agent
- Constrain Solving Problem: variables, domains, constraints

Evaluation of search strategies

- Basic questions
  - Will a search algorithm find a solution?
  - Which algorithm is the best one?

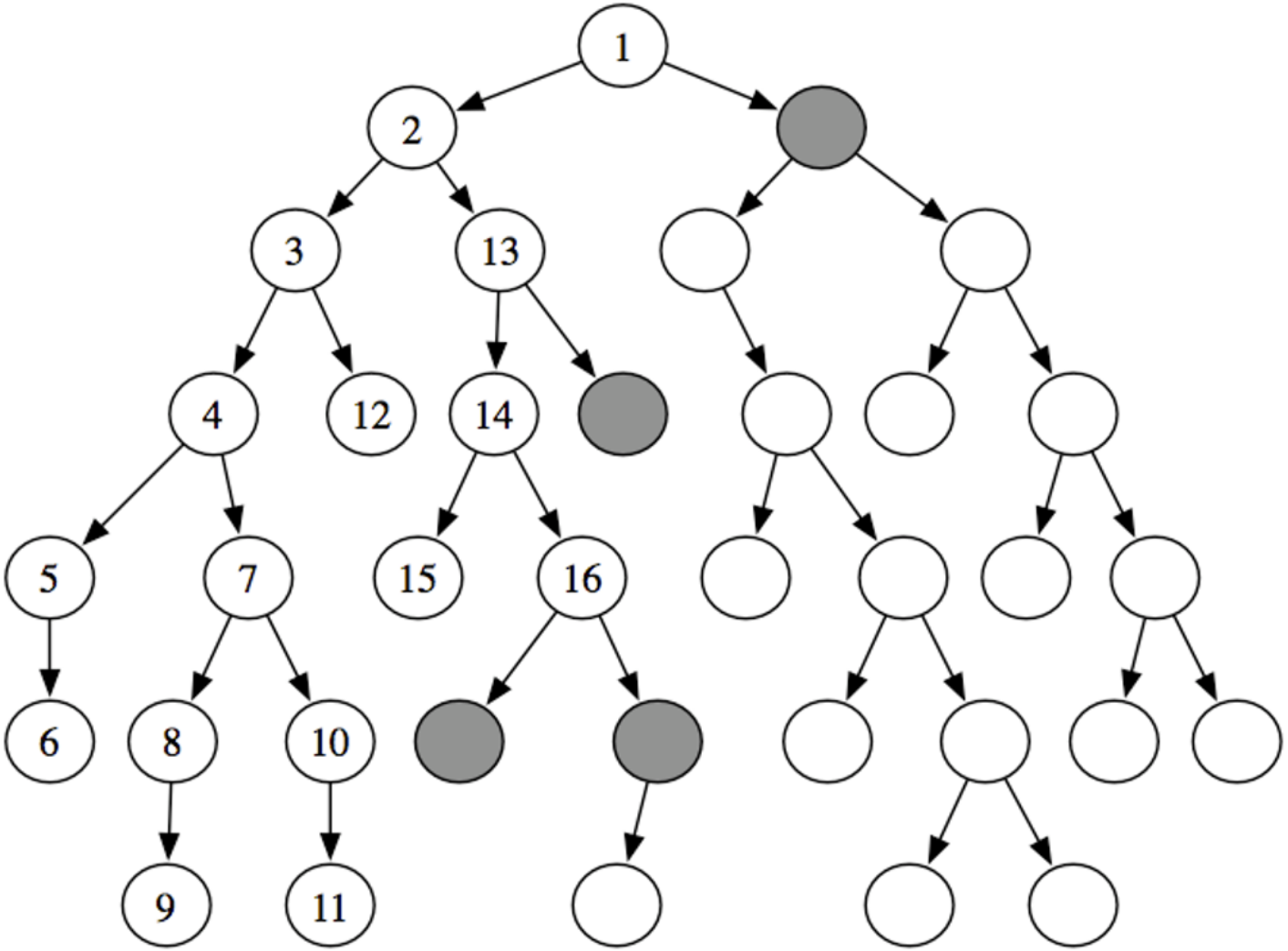
  cf Mike Chiang (2011) http://www.cs.ubc.ca/~hutter/teaching/cpsc322/2-Search3.pdf
- Completeness
  - whenever there is at least one solution, the algorithm is guaranteed to find it within a finite amount of time
- Optimality
  - a search algorithm is optimal if, when it finds a solution, it is the best one
- Complexity
  - temporal complexity in the worst case
    - the maximum path length *m*
    - the maximum forward branching factor *b* or an average (effective) branching factor
  - memory complexity in the worst case
    - the maximum number of nodes on the frontier

Depth-First Search



- Figure: The order of generating and analysing nodes
  - upper numbers – the order in which the nodes are analysed,
  - bottom – the order in which the nodes are generated,
  - double frame – goal node
- Properties
  - incomplete (sic!)
  - non-optimal (sic!)
- Complexity:
  - temporal O(bd),
  - memory O(db)
    - where *b* — branching factor, *d* — searching depth

Depth-First Search: ordering of nodes



http://artint.info/html/ArtInt_53.html

Iterative Deepening Depth First

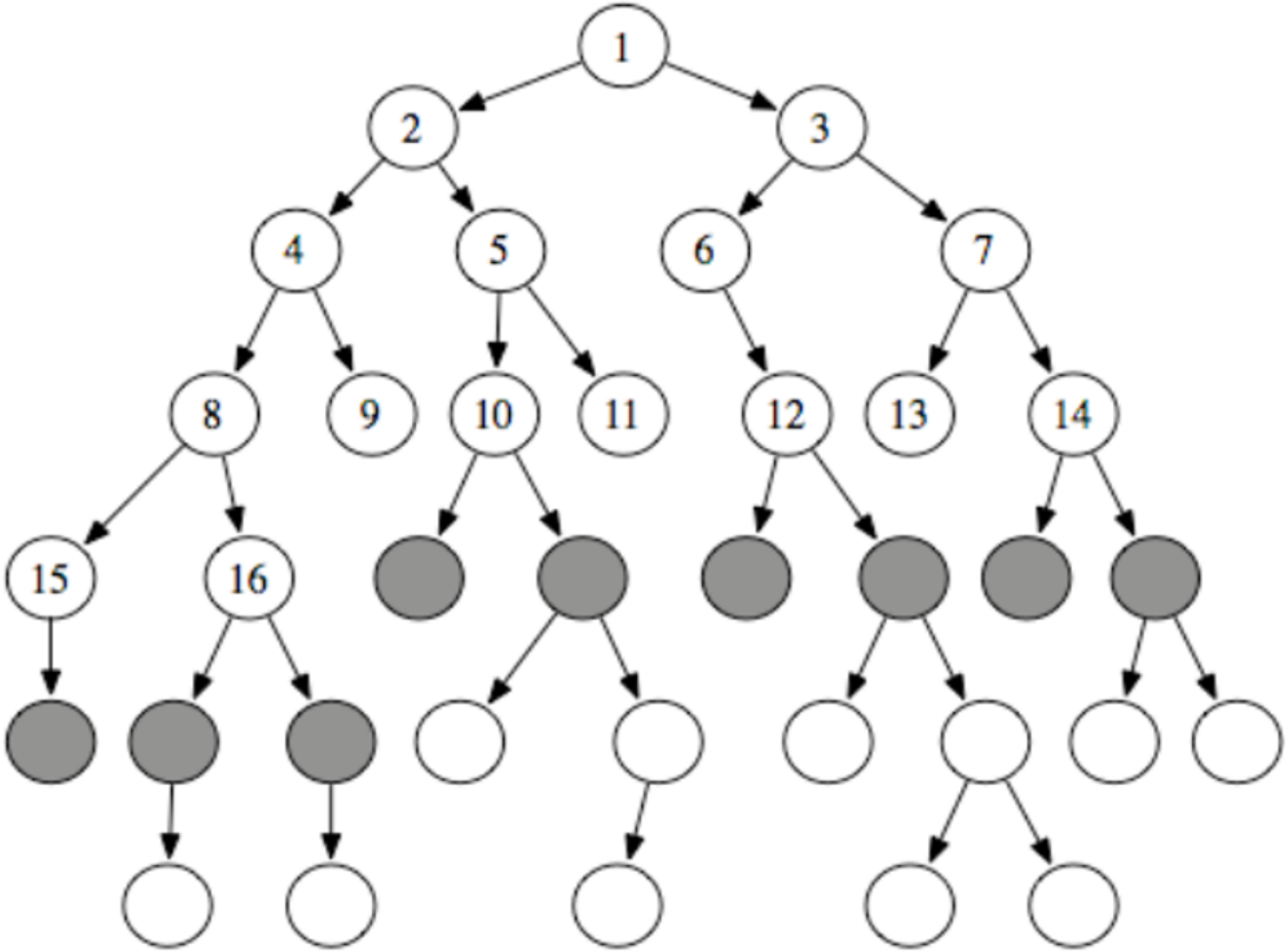
```
function ITERATIVE-DEEPENING-SEARCH( problem) returns a solution sequence
    inputs: problem, a problem

    for depth — 0 to ∞ do
      if DEPTH-LIMITED-SEARCH( problem, depth) succeeds then return its result
    end
    return failure
```

Figure 3.15 The iterative deepening search algorithm.

(Russel & Norvig 1995, pp. 79)

- Complete and optimal
- Complexity: temporal O(bd), memory O(db)
- For b=10 i d=5, 11% more nodes visited than DFS

Breadth-First Search

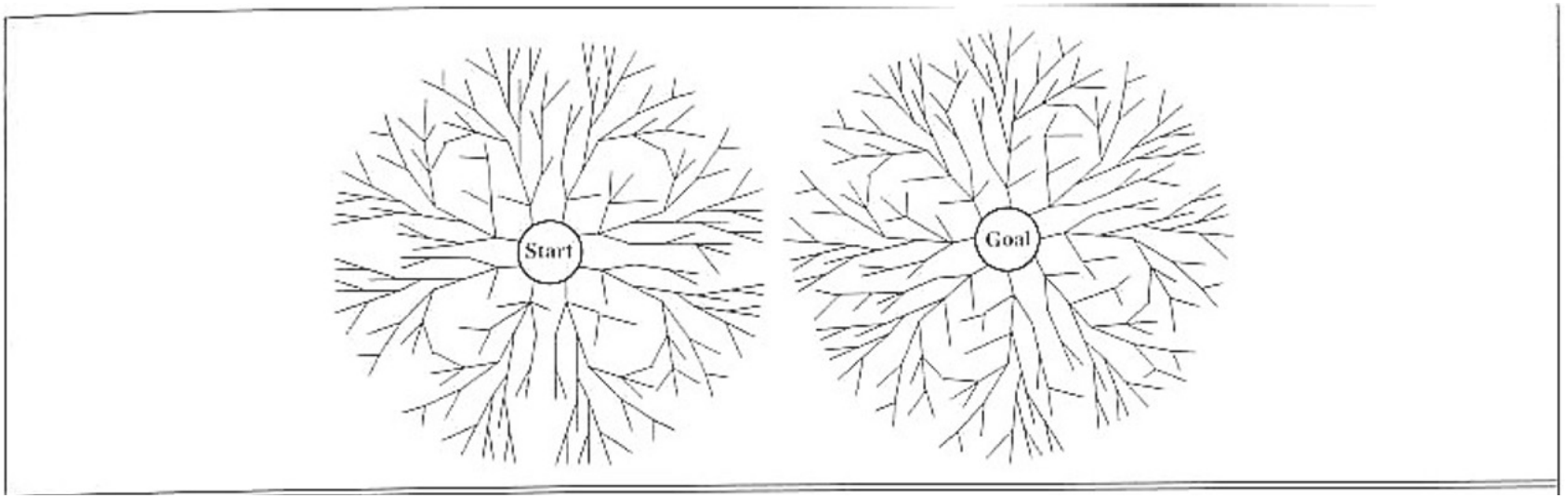


http://artint.info/html/ArtInt_54.html

Breadth-First Search

- Complete
- Optimal
  - under the condition of the non-decreasing cost in relation to the depth of searching
- Complexity:
  - temporal O(bd),
  - memory ??
    - *A reader may take a look at a couple of slides back!*

Bidirectional BFS



(Russel & Norvig 1995, pp. 81)

- Complexity: $O(2b^{d/2})=O(b^{d/2})$
  - e.g. for b=10, d=6, BFS expands 1,111,111 nodes, Bi-BFS 2,222
- Problems
  - reversible node expand operators
  - multiple target nodes
  - fast checking for meetings, e.g. a hash table

Lowest-Cost-First Search (Search with costs)

- Sometimes, a cost value is associated with each arc
- The path cost = the sum of the arc costs

$$\text{cost}(\langle n_0, \ldots, n_k \rangle) = \sum_{i=1}^{k} \text{cost}(\langle n_{i-1}, n_i \rangle)$$

- Here, we usually want to find a solution that minimises the cost
- It selects unvisited nodes with the least cost (e.g., distance), calculating the cost (distance) from it to each unvisited neighbour, updating their distance if it is smaller (initial distances, e.g., infinity)
- This method guarantees the optimum solution if:
  - a positive number limits the cost of arches
  - the branching factor is finite
  - the solution exists
- Moreover, the first path found is the cheapest (under conditions, e.g, cost>0)

Problems with blind search

- Expansion of an already visited state results in a loss
- Solution
  - blocking the generation of states identical to the source
    - one arc cycles
  - blocking the generation of cyclic paths
    - cycle detection is very costly in general
  - blocking the generation of any previously visited state – an advanced hashing is required

Heuristic search

- Heuristic information about which node is more 'promising' is provided by a heuristic function $h(n)$,
- For a given node n, h returns a non-negative real value, estimating the path cost from node n to a goal node
  - returns 0 for the goal node
- The function $h(n)$ is admissible, if $h(n)$ is less than or equal to the current smallest value of the actual cost function from the current node n to the goal node
- Heuristic function
  - a way of informing about the search direction towards a target,
  - providing information to guess which neighbouring nodes may lead to a target

Heuristics

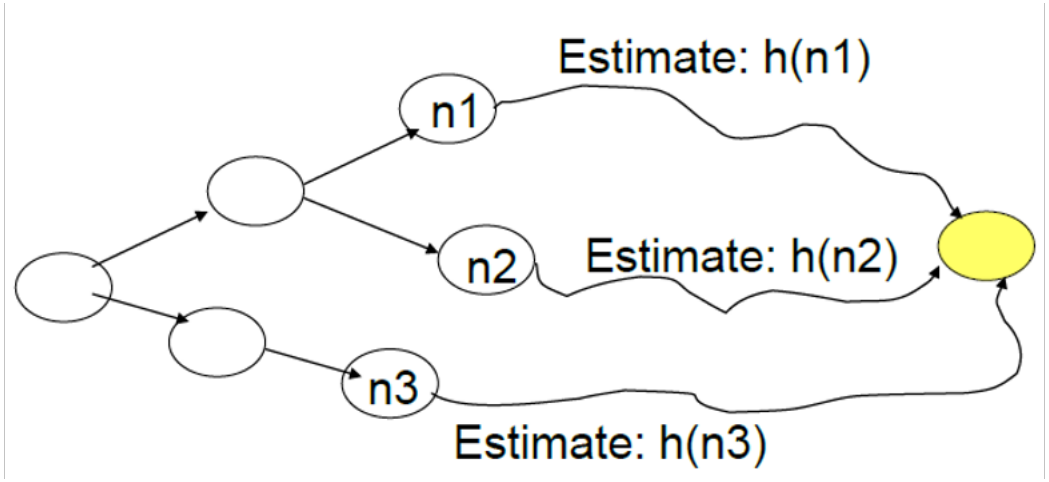
- Heuristics - the ancient Greek *heuriskein*, the art of finding solutions to problems or discovering new methods of solution
- Pappus from Alexandria, III / IV century
  - analysis - laying plan of solutions (decomposing the result into elements)
  - synthesis - the execution of the plan (outputting the result from the elements obtained from the analysis)
- Georgy Polya (1945): “How to solve it?” New York: Princeton
  - considered to be one of the most important in AI literature
  - dedicated to teaching mathematics, including teaching thinking, includes the "Short Heuristic Dictionary", which contains the basic concepts of heuristics
  - treated heuristics as a study and practical use, emphasising its experimental foundations
- Recently
  - understanding heuristics is more and more distant from the concept of Polya – his methods were mainly about exploring, allowing “blind actions”
  - solving with heuristics – it is essential to use methods closely related to the specificity of the problem domain
  - the dominant direction for introducing new heuristics is to improve existing methods and strategies

Heuristics in searching

- Features of heuristics
  - uncertainty of the outcome
  - incomplete knowledge available
  - improving the solution obtained
- The most crucial function of heuristics is to direct a decision-making process
  - by pointing to the best directions towards a solution
  - or indirectly, by eliminating the least promising directions
- Strategies are modified to design a quasi-optimal (instead of optimal) solution with significant cost reductions
- Better quality of the solution results from the application of practical rules (often intuitive and empirical), closely related to the domain of the problem

Best-First Search

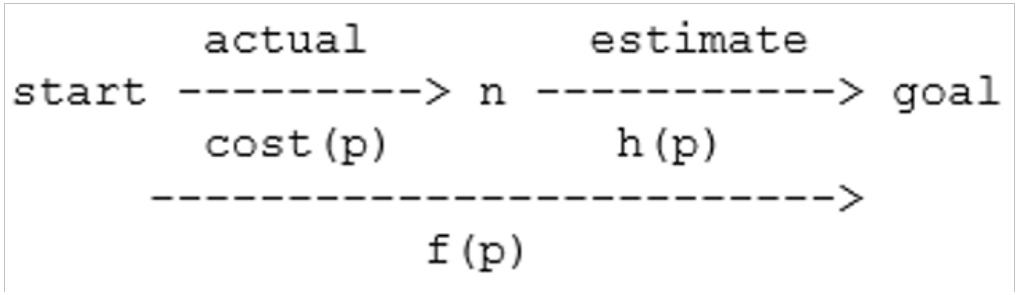
- Misleading name
- Greedy search:
  - select a node from the front with the smallest value of the heuristic function
  - non-complete and non-optimal
- It locally chooses the best path, but explores all the paths from a selected path before choosing another path



http://www.cs.utexas.edu/~mooney/cs343/slide-handouts/heuristic-search.4.pdf

A* search, A Star

- Combination of two strategies: lowest cost and best-first
- It considers both the cost of the path and heuristic information when choosing a node to expand
- For each node from the front, A* uses the estimated cost of the entire path from the start to a target
- Uses the cost of the hitherto found path $cost(p)$ and the heuristic function $h(p)$, which estimates the path cost from the end of the path to the goal
- For a node n on a path p:

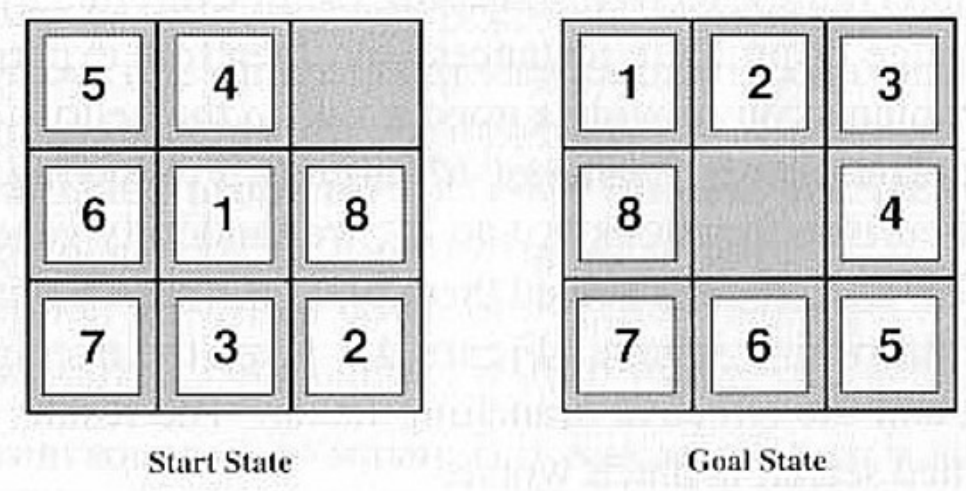


A* search, A Star

- Popular method (scheme)
- Its purpose is to find the cheapest route in the graph
- Evaluation function, heuristic = sum of two components:
  - *cost*(*w*) - defines the cost of the path connecting the starting node s to the node *w*,
  - *h*(*w*) - this is an estimate based on the heuristic information, the cost of a path connecting *w* to a target node (or the best target node)
- For $h=0$ and $cost(w_1,w_2)=1$, for all nodes w_1, w_2, BFS becomes the special case of strategy A*
- When the heuristic function *f* is selected as:
  - f(p)=0, f(b)=f(a)-1
  - where b is the direct descendant of node a, strategy A * becomes an "depth"

Heuristics in Searching

- Quality of heuristics
  - effective branching factor
  - $N = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$
  - e.g. A* finds solution in d=5 and N=52
  - effective branching factor $b^* = 1.91$
- Examples of heuristics
  - 8-blocks puzzle
  - Hamming — the number of blocks in the wrong position
  - Manhattan distance (city block distance)



Heuristics in Searching

- Contour
  - more directed, the better
- For admissible
  - f(n) < f*(n)
  - goals first
  - optimally effective A* for the given heuristics

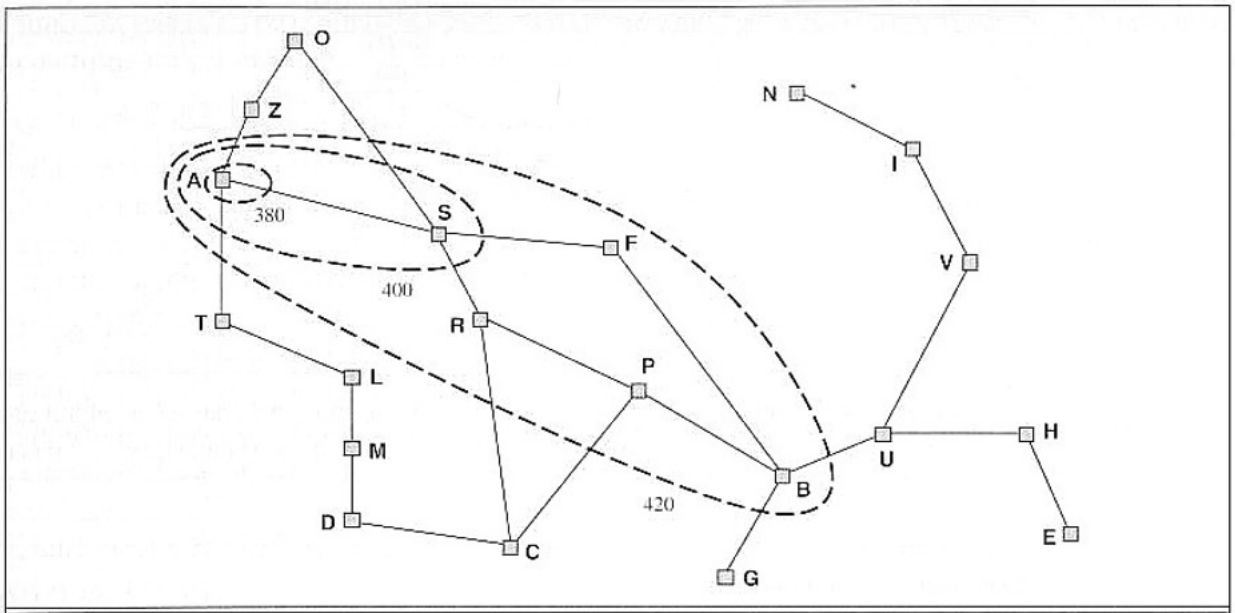


Figure 4.5 Map of Romania showing contours at $f = 380$, $f = 400$ and $f = 420$, with Arad as the start state. Nodes inside a given contour have f-costs lower than the contour value.

(Russel & Norvig 1995, pp. 99)

Heuristics in Searching

- Discovering a heuristic
  - problem relaxation
    - simplification or relaxation of constraints
    - dropping off one or several formal constraints
    - e.g. in the case of a block puzzle allowing for block exchange or diagonal moves
    - fast finding exact solution for a relaxed problem, Absolver (Prieditis 1993) – e.g. Rubik's cube
  - combination of admissible heuristics
    - admissible heuristic – never over-estimates, monotonically increasing together with the cost: $h(n) = \max(h_1(n), \ldots, h_m(n))$
- Discovering a heuristic (cont.)
  - linear combination of features,
    - e.g., selection of coefficients using Machine Learning or genetic algorithms.
  - correcting a heuristic on the basis of statistical information or machine learning
    - e.g. when h(n)=14, then in 90% of cases the real cost of goal reaching is 18
- Cost of computing a heuristic
  - it can very significantly influence the overall cost of searching

A* optimality

- A* admissibility, this is a proven property:
- A* always finds the optimal solution, and it is the first solution found if:
  - the solution exists,
  - the branching factor is finite (each node has a finite number of neighbours)
  - the cost of arcs is greater than some $\varepsilon>0$,
  - h(n) is the lower bound of the actual minimum cost of the path with the lowest cost from n to the target node
    - heuristic function is optimistic
- Note: the admissibility of A* does not ensure that each intermediate node chosen from the front is on an optimal path from start to destination

Memory-bounded Heuristic Search

- Origin:
  - very large memory complexity of A* (on average)
- Approaches:
  - Iterative Deepening A* (IDA*)
  - Simplified Memory-bounded A* (SMA*)

Iterative Deepening A*

- Idea
  - following the scheme of Iterative Deepening DFS
  - searching confined to more and more expanded contours of the total cost function f
- A larger set of f values worsens the properties of the algorithm
  - constant step of the contour deepening e
  - an e admissible solution
- Optimal and complete in the same sense as A*
- Memory complexity: bf*/d,
  - where b – a branching factor, f* – the optimal path cost, d – the lowest cost of an operator

Iterative Deepening A*

```
function IDA*(problem) returns a solution sequence
   static f-limit the current f-Cost limit
   static root    a node
{ root <-- Make-Node(INITIAL-STATE[problem])
  f-limit <-- f-Cost(root)
loop {
   solution,f-limit <-- DFS-CONTOUR(root,f-limit)
   if solution is non-null then return solution
   if f-limit is INFINITE then return failure }}

function DFS-CONTOUR(node,f-limit)
     returns a solution sequence and a new f-Cost limit
  static next-f the f-Cost limit for the next contour,
         initially INFINITE
{ if f-Cost[node]>f-limit then return null, f-Cost[node]
  if GOAL-TEST[problem](STATE[node])then return node,f-limit
  for each node s in SUCCESSOR(node) do
   {solution,new-f <-- DFS-CONTOUR(s,f-limit)
    if solution is non-null then return solution,f-limit
    next-f <-- MIN(next-f,new-f)}
  return null, next-f}
```

SMA*: Simplified Memory Bounded A*

- Origins
  - IDA* main drawback: constant repetition of searching the same states
  - idea: to use the available memory to keep the most promising partial paths
- SMA* properties
  - uses the available memory
  - avoids repeated expansion of states as long as it is allowed by the available memory
  - complete if the memory enables storing the shallowest solution (the shortest path in the graph)
  - optimal if the memory enables storing the shallowest solution
    - otherwise, it returns the best partial solution that is possible to be reached within the limits of the available memory
  - assuming an adequate amount of memory, the algorithm is optimally effective

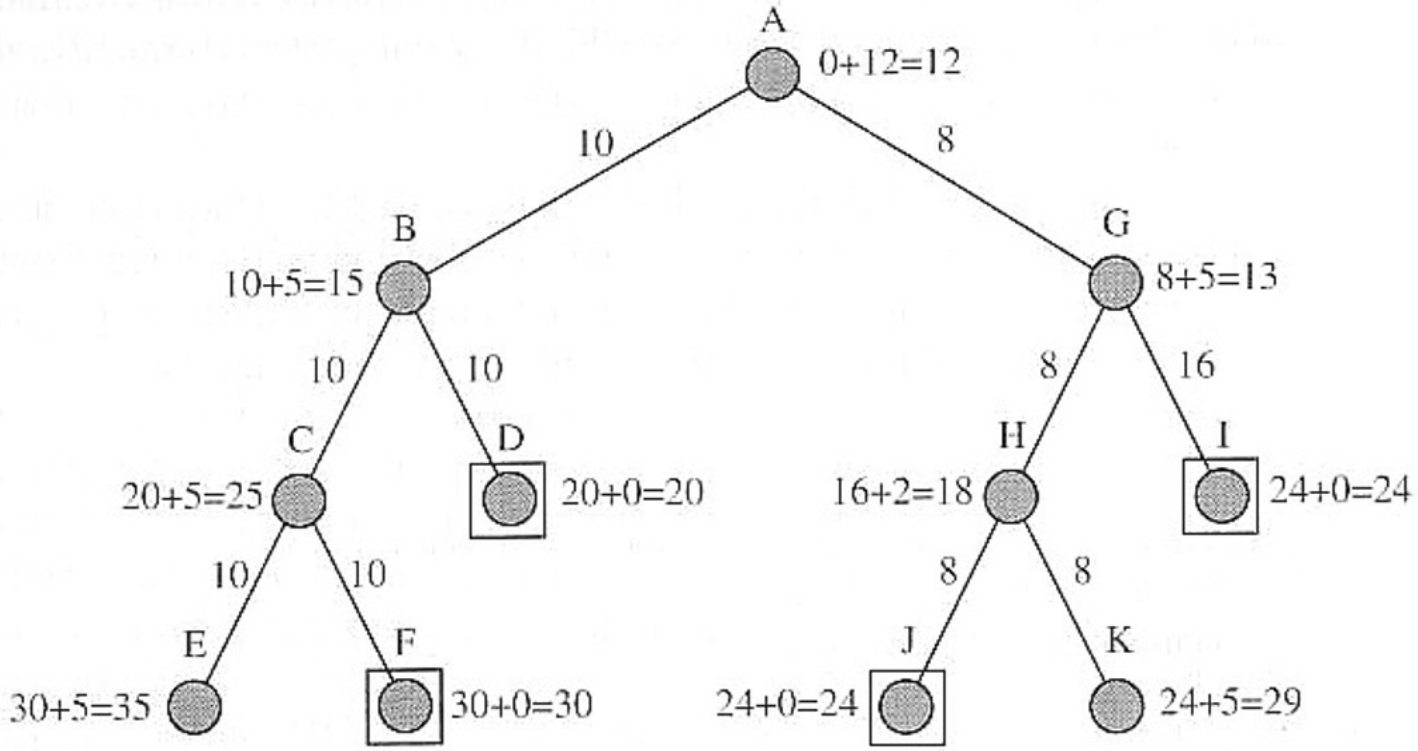
SMA*: Work

- After filling the memory, the shallowest nodes with the highest cost are forgotten (removed)
  - the cost of the forgotten node is stored in its ancestor
  - after exhausting the expansion possibilities, a forgotten node with the lowest cost is expanded

SMA*: Simplified Memory Bounded A*

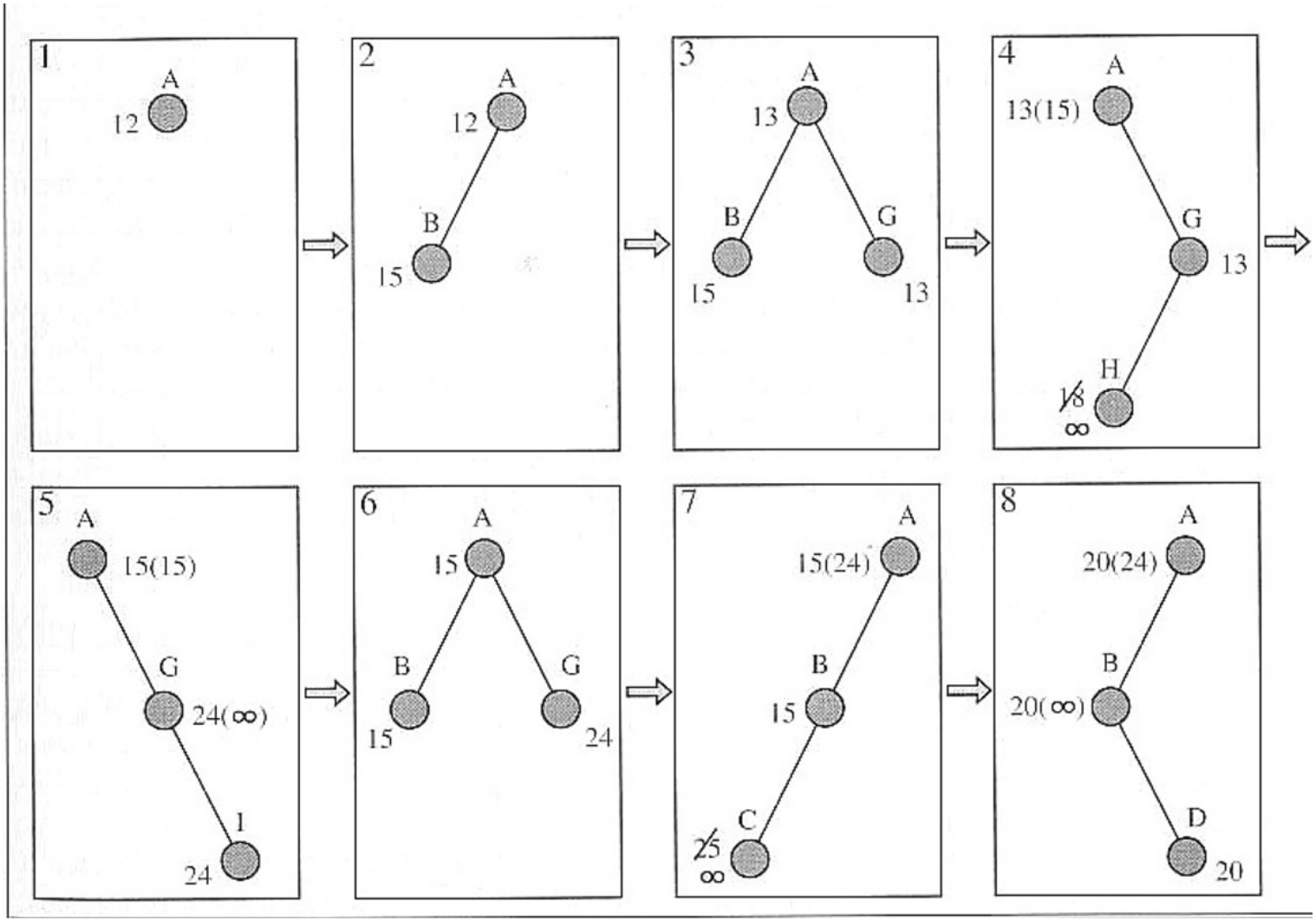
```
function SMA*(problem) returns a solution sequence
  static Queue a queue of nodes ordered by f-cost
{ Queue <-- MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[problem]))
  loop {
   if Queue is empty then return failure
   n <-- deepest least f-cost node in Queue
   if GOAL-TEST(n) then return success
   s <-- NEXT-SUCCESSOR(n)
   if s is not a goal and is at maximum depth then
       f(s) <- infinite_value
   else f(s) <-- MAX(f(n),g(s)+h(s))
   if all of n's successors have been generated then
    update n's f-cost
              and those of its ancestors if necessary
   if SUCCESSORS(n) all in memory then remove n from Queue
   if memory is full then
    {delete shallowest, highest f-cost node in Queue
     remove it from its parent's successor list
     insert its parent on Queue if necessary}
   insert s in Queue}  }
```

SMA*: Example



- Rectangles — goals
- Memory size: 3 nodes
- Each node is described with the cost at a given moment
- Values in the brackets: the best 'forgotten' descendant

SMA*: Example


1
A
12
2
A
12
B
15
3
A
13
B
G
15
13
4
A
13(15)
G
13
H
1/8
∞
5
A
15(15)
G
24(∞)
1
24
6
A
15
B
G
15
24
7
A
15(24)
B
15
C
2/5
∞
8
A
20(24)
B
20(∞)
D
20

A* Applications – heuristic search applications

- Road planning
  - Map,
  - Virtual spaces, e.g. in computer games,
  - Computer networks.
- Finding optimal paths in complex graphs
  - E.g. representing the space of knowledge, the space of concepts,
  - E.g. determining various measures of similarity based on knowledge graphs, etc.

Local Search

- Not all problems require finding a complete path of actions (moves, etc.).
  - achieving a goal state is enough.
- Evaluation of states
  - objective function — how good a state is from the goal perspective
  - measure
    - distance — e.g. the number of changes to introduce,
    - similarity — e.g. number of matching features
    - difference — in the value of the objective function
- Loss instead of a goal
  - closeness to the goal states
  - loss for the goal states: zero
  - based on or related to the objective function
  - often heuristics
- Local search
  - the algorithm depends only on the local characteristics of states
  - state-specific loss

Hill Climbing

Algorithm HillClimb(Initial State: s, Loss function: L(.))
begin
  CURRENT= {s};
  repeat
    Scan adjacent states to CURRENT until all states are scanned or a state NEXT is found with lower loss than CURRENT;
    if a state NEXT was found with lower loss than that CURRENT
      set CURRENT=NEXT;
    until no improvement in previous iteration;
    return CURRENT;
end

- Maximisation/minimisation centric objective function
  - hill climbing — `going towards increasing values'
- Local selection of states towards improvement
- Main problem: local optimum
  - not complete
  - prone to stacking in local maxima (minima) - no transition from it
- Scanning adjacent states
  - all (maximal/minimal), first larger/smaller, random

Tabu Search

Algorithm TabuSearch(Initial State: s, Loss function: L(.))
begin
CURRENT= {s};
VISIT= { }; {VISIT is the Tabu [taboo] list }
repeat
Add CURRENT to VISIT:
if all neighbours of CURRENT are in VISIT **then** set RANDOM= **nil**;
else set RANDOM to a random neighbour of CURRENT that is not in VISIT;
Scan adjacent states to CURRENT until all states are scanned or a state NEXT $\notin$ VISIT is found with lower loss than CURRENT;
if a state NEXT $\notin$ VISIT was found with lower loss than that of CURRENT
then set CURRENT=NEXT
else if RANDOM$\neq$**nil** set CURRENT= RANDOM;
until RANDOM= **nil**;
return CURRENT;
end

Tabu Search

- Allows the current solutions to worsen to allow escaping from a local optimum
  - local optimum — only states already visited are better
  - taboo states
  - random `jump' to a non-visited state
- Modifications
  - the best possible neighbour is selected
    - quicker improvement, but lower efficiency
  - limited size of the VISITED hash table
    - earliest nodes removed first
  - various schemes of `short-term' and `long-term' memories